R Programming Practical Examination

Vince Ryan Del Rosario

November 11, 2026

Part A: Basic Operations

A1 Create `v` as the sequence 5, 10, 15, . . . , 50 using `seq()`. Print its length, sum and mean.

```
v <- seq(5, 50, by = 5)
v
```

```
##  [1]  5 10 15 20 25 30 35 40 45 50
```

```
length(v)
```

```
## [1] 10
```

```
sum(v)
```

```
## [1] 275
```

```
mean(v)
```

```
## [1] 27.5
```

A2 Create `w` that repeats the vector `c(1, 2, 3)` four times, and `z` that repeats each element of `c(1, 2, 3)` four times. Print both.

```
w <- rep(c(1, 2, 3), times = 4)
z <- rep(c(1, 2, 3), each = 4)
w
```

```
##  [1] 1 2 3 1 2 3 1 2 3 1 2 3
```

```
z
```

```
##  [1] 1 1 1 1 2 2 2 2 3 3 3 3
```

A3 Compute and print: the remainder of 17 divided by 5, the integer division of 17 by 5, 2 to the power of 10, the square root of 144, and pi rounded to 3 decimals.

```r
17 %% 5
```

```
## [1] 2
```

```r
17 %/% 5
```

```
## [1] 3
```

```r
2^10
```

```
## [1] 1024
```

```r
sqrt(144)
```

```
## [1] 12
```

```r
round(pi, 3)
```

```
## [1] 3.142
```

A4 Given `a <- c(2, 4, 6, 8)` and `b <- c(1, 3, 5, 7)`, print `a + b`, `a * b`, the logical comparison `a > b`, and the dot product `sum(a * b)`.

```r
a <- c(2, 4, 6, 8)
b <- c(1, 3, 5, 7)
a + b
```

```
## [1]  3  7 11 15
```

```r
a * b
```

```
## [1]  2 12 30 56
```

```r
a > b
```

```
## [1] TRUE TRUE TRUE TRUE
```

```r
sum(a * b)
```

```
## [1] 100
```

A5 Use `paste()` to produce "Student-1" "Student-2" "Student-3". Then identify the datatype of 42, 42L, "42", 411i, and TRUE. What is its output?

```r
paste("Student", 1:3, sep = "-")
```

```
## [1] "Student-1" "Student-2" "Student-3"
```

```r
class(42)
```

```
## [1] "numeric"
```

```r
class(42L)
```

```
## [1] "integer"
```

```r
class("42")
```

```
## [1] "character"
```

```r
class(411i)   # if the 4th value on your paper is plain 411, use class(411)
```

```
## [1] "complex"
```

```r
class(TRUE)
```

```
## [1] "logical"
```

```r
# Output of paste: "Student-1" "Student-2" "Student-3"
# Types: numeric, integer, character, complex, logical
```

Part B: Data Frames and Conditional Statements

B1 Create a data frame `students` with 8 rows and these columns (use `stringsAsFactors = FALSE`; sex is "M" or "F"):

```r
students <- data.frame(
  id         = 1:8,
  name       = c("Alice","Bruno","Chen","Dana","Elif","Farid","Grace","Hiro"),
  sex        = c("F","M","M","F","F","M","F","M"),
  section    = c("A","B","A","C","B","C","A","B"),
  midterm    = c(78, 85, 92, 67, 74, 88, 95, 59),
  final      = c(82, 79, 96, 71, 80, 91, 93, 65),
  attendance = c(95, 88, 100, 72, 90, 85, 98, 60),
  stringsAsFactors = FALSE
)
students
```

```
##   id  name sex section midterm final attendance
## 1  1 Alice   F       A      78    82         95
## 2  2 Bruno   M       B      85    79         88
## 3  3  Chen   M       A      92    96        100
## 4  4  Dana   F       C      67    71         72
## 5  5  Elif   F       B      74    80         90
## 6  6 Farid   M       C      88    91         85
## 7  7 Grace   F       A      95    93         98
## 8  8  Hiro   M       B      59    65         60
```

B2 Display the structure, dimensions, column names, and summary of `students`.

```
str(students)
```

```
## 'data.frame':    8 obs. of  7 variables:
##  $ id        : int  1 2 3 4 5 6 7 8
##  $ name      : chr  "Alice" "Bruno" "Chen" "Dana" ...
##  $ sex       : chr  "F" "M" "M" "F" ...
##  $ section   : chr  "A" "B" "A" "C" ...
##  $ midterm   : num  78 85 92 67 74 88 95 59
##  $ final     : num  82 79 96 71 80 91 93 65
##  $ attendance: num  95 88 100 72 90 85 98 60
```

```
dim(students)
```

```
## [1] 8 7
```

```
names(students)
```

```
## [1] "id"         "name"       "sex"        "section"    "midterm"
## [6] "final"      "attendance"
```

```
summary(students)
```

```
##        id              name               sex             section     midterm
##  Min.   :1.00   Length   :8   Length   :8   Length   :8   Min.   :59.00
##  1st Qu.:2.75   N.unique :8   N.unique :2   N.unique :3   1st Qu.:72.25
##  Median :4.50   N.blank  :0   N.blank  :0   N.blank  :0   Median :81.50
##  Mean   :4.50   Min.nchar:4   Min.nchar:1   Min.nchar:1   Mean   :79.75
##  3rd Qu.:6.25   Max.nchar:5   Max.nchar:1   Max.nchar:1   3rd Qu.:89.00
##  Max.   :8.00                                            Max.   :95.00
##      final         attendance
##  Min.   :65.00   Min.   : 60.00
##  1st Qu.:77.00   1st Qu.: 81.75
##  Median :81.00   Median : 89.00
##  Mean   :82.12   Mean   : 86.00
##  3rd Qu.:91.50   3rd Qu.: 95.75
##  Max.   :96.00   Max.   :100.00
```

B3 Add a column `average` = the mean of `midterm` and `final` for each student, rounded to 1 decimal.

```
students$average <- round((students$midterm + students$final) / 2, 1)
students
```

```
##   id  name sex section midterm final attendance average
## 1  1 Alice   F       A      78    82         95    80.0
## 2  2 Bruno   M       B      85    79         88    82.0
## 3  3  Chen   M       A      92    96        100    94.0
## 4  4  Dana   F       C      67    71         72    69.0
## 5  5  Elif   F       B      74    80         90    77.0
## 6  6 Farid   M       C      88    91         85    89.5
## 7  7 Grace   F       A      95    93         98    94.0
## 8  8  Hiro   M       B      59    65         60    62.0
```

B4 Add a logical column `passed` that is TRUE when `average >= 70` **and** `attendance >= 75`.

```
students$passed <- students$average >= 70 & students$attendance >= 75
students
```

```
##   id  name sex section midterm final attendance average passed
## 1  1 Alice   F       A      78    82         95    80.0   TRUE
## 2  2 Bruno   M       B      85    79         88    82.0   TRUE
## 3  3  Chen   M       A      92    96        100    94.0   TRUE
## 4  4  Dana   F       C      67    71         72    69.0  FALSE
## 5  5  Elif   F       B      74    80         90    77.0   TRUE
## 6  6 Farid   M       C      88    91         85    89.5   TRUE
## 7  7 Grace   F       A      95    93         98    94.0   TRUE
## 8  8  Hiro   M       B      59    65         60    62.0  FALSE
```

B5 Using `ifelse()`, add a column `standing` with these values based on `average`:

- "Honors" if average >= 90
- "Good" if average >= 75 (and below 90)
- "Needs Support" otherwise

```
students$standing <- ifelse(students$average >= 90, "Honors",
                     ifelse(students$average >= 75, "Good", "Needs Support"))
students
```

```
##   id  name sex section midterm final attendance average passed      standing
## 1  1 Alice   F       A      78    82         95    80.0   TRUE          Good
## 2  2 Bruno   M       B      85    79         88    82.0   TRUE          Good
## 3  3  Chen   M       A      92    96        100    94.0   TRUE        Honors
## 4  4  Dana   F       C      67    71         72    69.0  FALSE Needs Support
## 5  5  Elif   F       B      74    80         90    77.0   TRUE          Good
## 6  6 Farid   M       C      88    91         85    89.5   TRUE          Good
## 7  7 Grace   F       A      95    93         98    94.0   TRUE        Honors
## 8  8  Hiro   M       B      59    65         60    62.0  FALSE Needs Support
```

B6 Write a function `classify(score)` that uses an if / else if / else chain and returns "Distinction" for 85 or above, "Merit" for 70 to 84, "Pass" for 50 to 69, and "Fail" below 50. Test it with `sapply(c(90, 73, 55, 40), classify)` and print the result.

```
classify <- function(score) {
  if (score >= 85) {
    "Distinction"
  } else if (score >= 70) {
    "Merit"
  } else if (score >= 50) {
    "Pass"
  } else {
    "Fail"
  }
}
result <- sapply(c(90, 73, 55, 40), classify)
print(result)
```

```
## [1] "Distinction" "Merit"       "Pass"        "Fail"
```

```
# "Distinction" "Merit" "Pass" "Fail"
```

Part C: rbind() and cbind()

C1 Create a data frame `new_students` with two students. Its columns must match the original, including `average`, `passed` and `standing`, computed the same way:

- id 9, Ines, F, section C, midterm 81, final 84, attendance 92
- id 10, Jamal, M, section A, midterm 70, final 68, attendance 80

Use `rbind()` to append them to `students`, saving the result as `students_all`.

```
new_students <- data.frame(
  id         = c(9, 10),
  name       = c("Ines", "Jamal"),
  sex        = c("F", "M"),
  section    = c("C", "A"),
  midterm    = c(81, 70),
  final      = c(84, 68),
  attendance = c(92, 80),
  stringsAsFactors = FALSE
)
new_students$average  <- round((new_students$midterm + new_students$final) / 2, 1)
new_students$passed   <- new_students$average >= 70 & new_students$attendance >= 75
new_students$standing <- ifelse(new_students$average >= 90, "Honors",
                         ifelse(new_students$average >= 75, "Good", "Needs Support"))

students_all <- rbind(students, new_students)
students_all
```

```
##    id  name sex section midterm final attendance average passed      standing
## 1   1 Alice   F       A      78    82         95    80.0   TRUE          Good
## 2   2 Bruno   M       B      85    79         88    82.0   TRUE          Good
## 3   3  Chen   M       A      92    96        100    94.0   TRUE        Honors
## 4   4  Dana   F       C      67    71         72    69.0  FALSE Needs Support
## 5   5  Elif   F       B      74    80         90    77.0   TRUE          Good
## 6   6 Farid   M       C      88    91         85    89.5   TRUE          Good
## 7   7 Grace   F       A      95    93         98    94.0   TRUE        Honors
## 8   8  Hiro   M       B      59    65         60    62.0  FALSE Needs Support
## 9   9  Ines   F       C      81    84         92    82.5   TRUE          Good
## 10 10 Jamal   M       A      70    68         80    69.0  FALSE Needs Support
```

C2 Create the numeric vector `quiz <- c(8, 7, 10, 6, 7, 9, 10, 5, 8, 6)`. Use `cbind()` to attach it to `students_all` as a column named `quiz`. Confirm the result is still a data frame with `class()`.

```
quiz <- c(8, 7, 10, 6, 7, 9, 10, 5, 8, 6)
students_all <- cbind(students_all, quiz = quiz)
students_all
```

```
##    id  name sex section midterm final attendance average passed      standing
## 1   1 Alice   F       A      78    82         95    80.0   TRUE          Good
## 2   2 Bruno   M       B      85    79         88    82.0   TRUE          Good
## 3   3  Chen   M       A      92    96        100    94.0   TRUE        Honors
## 4   4  Dana   F       C      67    71         72    69.0  FALSE Needs Support
## 5   5  Elif   F       B      74    80         90    77.0   TRUE          Good
## 6   6 Farid   M       C      88    91         85    89.5   TRUE          Good
## 7   7 Grace   F       A      95    93         98    94.0   TRUE        Honors
## 8   8  Hiro   M       B      59    65         60    62.0  FALSE Needs Support
## 9   9  Ines   F       C      81    84         92    82.5   TRUE          Good
## 10 10 Jamal   M       A      70    68         80    69.0  FALSE Needs Support
##    quiz
## 1     8
## 2     7
## 3    10
## 4     6
## 5     7
## 6     9
## 7    10
## 8     5
## 9     8
## 10    6
```

```r
class(students_all)   # "data.frame"
```

```
## [1] "data.frame"
```

C3 Create the matrix `m <- cbind(midterm = students_all$midterm, final = students_all$final)` and use `rbind()` to append a final row containing the column means, labelled "mean".

```r
m <- cbind(midterm = students_all$midterm, final = students_all$final)
m <- rbind(m, mean = colMeans(m))
m
```

```
##      midterm final
##         78.0  82.0
##         85.0  79.0
##         92.0  96.0
##         67.0  71.0
##         74.0  80.0
##         88.0  91.0
##         95.0  93.0
##         59.0  65.0
##         81.0  84.0
##         70.0  68.0
## mean    78.9  80.9
```

C4 In a comment (2-3 sentences), explain what error `rbind()` produces if the data frames have different column names, and show the output if you safely catch it with `tryCatch()`.

```r
# rbind() on data frames requires the same column names (and number of columns).
# If the names differ, it stops with an error such as "names do not match
```

```r
# previous names", so the frames cannot be stacked.
bad <- data.frame(id = 11, fullname = "Zed", stringsAsFactors = FALSE)
tryCatch(rbind(students, bad),
         error = function(e) {
           message("rbind failed: ", conditionMessage(e))
           NULL
         })
```

```
## rbind failed: numbers of columns of arguments do not match
```

```
## NULL
```

Part D: Subsetting

Use `students_all` (with `quiz`) for all questions. Print every result.

D1 Extract the `name` column in three ways: `$`, `[["name"]]`, and `[ , "name"]`.

```r
students_all$name
```

```
##  [1] "Alice" "Bruno" "Chen"  "Dana"  "Elif"  "Farid" "Grace" "Hiro"  "Ines"
## [10] "Jamal"
```

```r
students_all[["name"]]
```

```
##  [1] "Alice" "Bruno" "Chen"  "Dana"  "Elif"  "Farid" "Grace" "Hiro"  "Ines"
## [10] "Jamal"
```

```r
students_all[, "name"]
```

```
##  [1] "Alice" "Bruno" "Chen"  "Dana"  "Elif"  "Farid" "Grace" "Hiro"  "Ines"
## [10] "Jamal"
```

D2 Extract rows 2 to 5 and only the columns `name`, `midterm`, `final`.

```r
students_all[2:5, c("name", "midterm", "final")]
```

```
##    name midterm final
## 2 Bruno      85    79
## 3  Chen      92    96
## 4  Dana      67    71
## 5  Elif      74    80
```

D3 Using logical subsetting, select students with `attendance` above 90.

```r
students_all[students_all$attendance > 90, ]
```

```
##   id  name sex section midterm final attendance average passed standing quiz
## 1  1 Alice   F       A      78    82         95    80.0   TRUE     Good    8
## 3  3  Chen   M       A      92    96        100    94.0   TRUE   Honors   10
## 7  7 Grace   F       A      95    93         98    94.0   TRUE   Honors   10
## 9  9  Ines   F       C      81    84         92    82.5   TRUE     Good    8
```

D4 Using `subset()`, select students of sex "F" in section "A" or "C" whose `final` is at least 75; return only `name`, `section` and `final`.

```
subset(students_all,
       sex == "F" & section %in% c("A", "C") & final >= 75,
       select = c(name, section, final))
```

```
##    name section final
## 1 Alice       A    82
## 7 Grace       A    93
## 9  Ines       C    84
```

D5 Use `which.max()` and `which.min()` to report the name of the student with the highest and lowest `average`.

```
students_all$name[which.max(students_all$average)]   # highest
```

```
## [1] "Chen"
```

```
students_all$name[which.min(students_all$average)]   # lowest
```

```
## [1] "Hiro"
```

D6 Sort `students_all` by `average` in descending order, then print the top 3 rows.

```
sorted <- students_all[order(students_all$average, decreasing = TRUE), ]
head(sorted, 3)
```

```
##   id  name sex section midterm final attendance average passed standing quiz
## 3  3  Chen   M       A      92    96        100    94.0   TRUE   Honors   10
## 7  7 Grace   F       A      95    93         98    94.0   TRUE   Honors   10
## 6  6 Farid   M       C      88    91         85    89.5   TRUE     Good    9
```

Part E: Basic Factors

E1 Convert `section` and `sex` in `students_all` to factors with `factor()`. Print `levels()` of each.

```
students_all$section <- factor(students_all$section)
students_all$sex     <- factor(students_all$sex)
levels(students_all$section)
```

```
## [1] "A" "B" "C"
```

```r
levels(students_all$sex)
```

```
## [1] "F" "M"
```

E2 Use `table()` to count students per `sex` and per `section`. Try the two-way table of sex by section: `table(students_all$sex, students_all$section)`. What is the output?

```r
table(students_all$sex)
```

```
##
## F M
## 5 5
```

```r
table(students_all$section)
```

```
##
## A B C
## 4 3 3
```

```r
table(students_all$sex, students_all$section)
```

```
##
##     A B C
##   F 2 1 2
##   M 2 2 1
```

```r
# Output of the two-way table: rows = sex (F, M), columns = section (A, B, C)
#      A B C
#   F  2 1 2
#   M  2 2 1
```

E3 Print `as.integer()` of the `sex` factor. In a comment, explain what the integers represent.

```r
as.integer(students_all$sex)
```

```
##  [1] 1 2 2 1 1 2 1 2 1 2
```

```r
# The integers are the internal codes of the factor levels:
# 1 = "F" and 2 = "M" (levels are sorted alphabetically).
```

E4 Create a new factor `sex_label` from `sex` using `factor()` with `levels = c("F", "M")` and `labels = c("Female", "Male")`. Add it to `students_all` and print its `levels()`.

```r
students_all$sex_label <- factor(students_all$sex,
                                 levels = c("F", "M"),
                                 labels = c("Female", "Male"))
levels(students_all$sex_label)
```

```
## [1] "Female" "Male"
```

E5 Print `summary()` of `sex_label`. Then, using logical subsetting and `mean()`, print the mean `average` for each level of `sex_label`.

```
summary(students_all$sex_label)
```

```
## Female   Male
##      5      5
```

```
mean(students_all$average[students_all$sex_label == "Female"])
```

```
## [1] 80.5
```

```
mean(students_all$average[students_all$sex_label == "Male"])
```

```
## [1] 79.3
```